CaregiverConnect: Families without Borders

User & Technical Guide

A full-stack community platform for single parents, guardians, and grandparents.

Featuring community discussions, curated resources, and AI-powered support chat.

Table of Contents

1. Overview

2. Landing Page

3. Sign Up

4. Login

5. Home Page – Community Members

6. Profile Management

7. Discussions

   7.1 Browsing Topics and Posts

   7.2 Creating Posts and Replies

   7.3 Topic Requests (Users)

   7.4 Admin: Managing Topics and Requests

8. Resources

9. Support Chat (AI-Powered)

   9.1 How to Use

   9.2 Chat Interface Elements
   9.3 How the AI Works Internally

   9.4 Embedding Generation (Auto)

9.5 Chat History & Session Storage

9.6 Example Questions

10. Technical Reference

10.1 Architecture Overview

10.2 AI / RAG System

10.3 Data Models

10.4 REST API Reference

10.5 Environment Variables

10.6 Running the Application

10.7 Authorization Model

1. Overview

CaregiverConnect: Families without Borders is a web application designed to support single parents, guardians, and grandparents by providing a safe community space, curated resources, and an AI-powered assistant that draws on community knowledge to answer questions.

1.1 Key Features

- Community forum with discussion topics, posts, replies, and likes
- Curated resource library with links to external content
- AI support chat powered by Google Gemini with Retrieval-Augmented Generation (RAG)
- User profile management with privacy controls
- Topic request workflow for community-driven content moderation
- Admin tools for managing topics, resources, and user requests

1.2 Technology Stack

Component	**Technology**
Frontend	React 19, Vite 6, Tailwind CSS 4
Backend	Node.js, Express.js
Database	MongoDB Atlas (Mongoose 8)
AI / LLM	Google Gemini 2.5 Flash
Embeddings	Xenova/all-MiniLM-L6-v2 (384 dimensions, runs in Node.js)
Reranking	Xenova/bge-reranker-base cross-encoder
Chat History Compaction	Groq llama-3.1-8b-instant (optional)
Authentication	bcryptjs password hashing, localStorage session

1.3 Application Routes

URL Path	Page
/	Landing page – public marketing page
/signup	Account registration
/login	User login
/home	Main authenticated view (tabs: Home, Discussions, Resources, Support Chat)

2. Landing Page

URL Path: /

The landing page is the public-facing entry point of the application. It introduces CaregiverConnect to new visitors and directs them to sign up or log in.

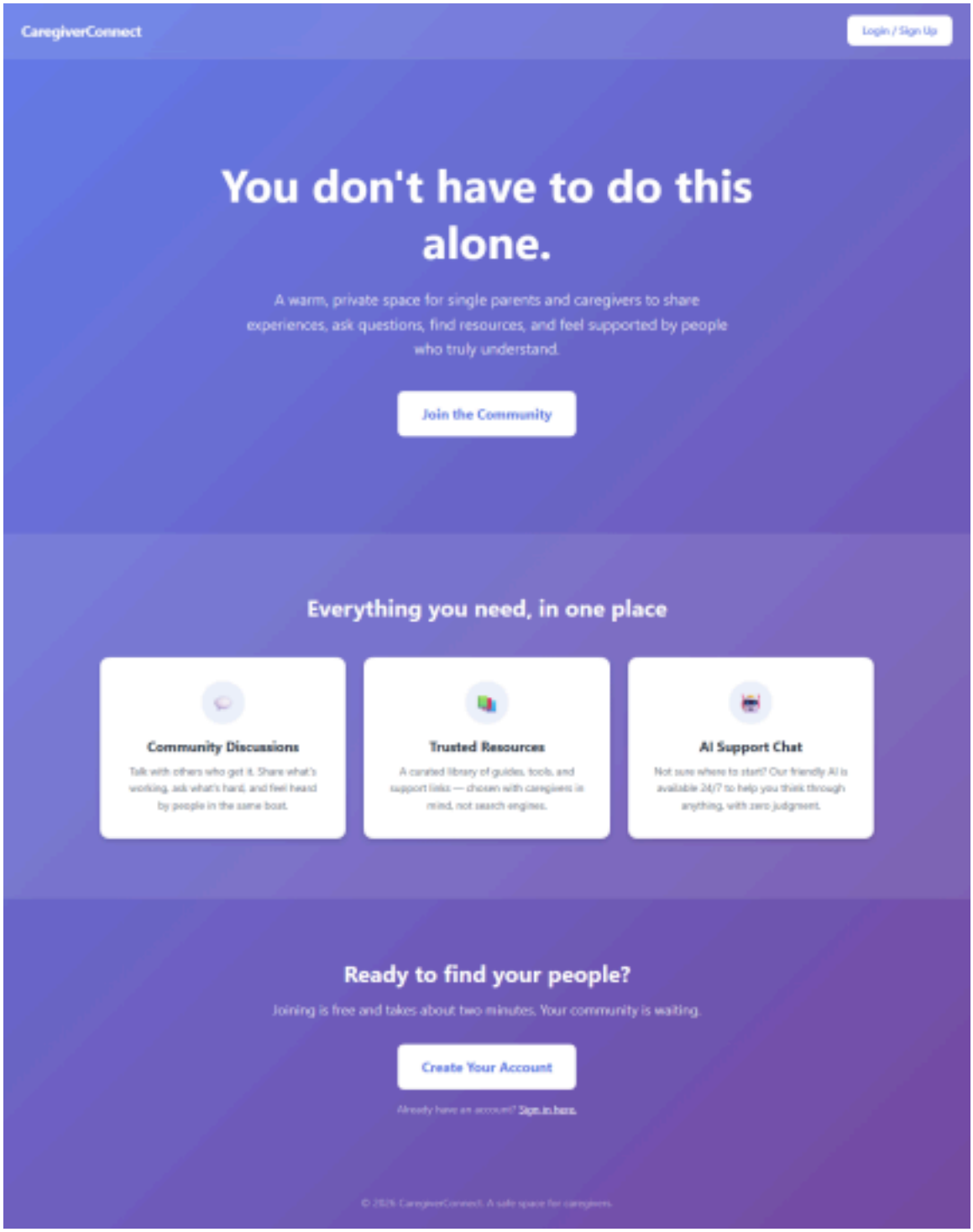


Figure 1: Landing Page – full view

2.1 Page Sections

Navigation Bar: Displays the app name/logo and a 'Login' link in the top-right corner.

Hero Section: Large heading introducing the platform with a tagline and two call-to-action buttons: 'Get Started' (links to /signup) and 'Log In' (links to /login).

Feature Cards: Three cards summarising the main features: Community Discussions, Trusted Resources, and AI Support Chat. Each card has an icon, title, and short description.

2.2 Navigation

Get Started button: Navigates to /signup

Log In button: Navigates to /login

Login link (navbar): Navigates to /login

3. Sign Up

URL Path: /signup

The sign-up page allows new users to create an account. All fields are validated on both the client and server side before the account is created.

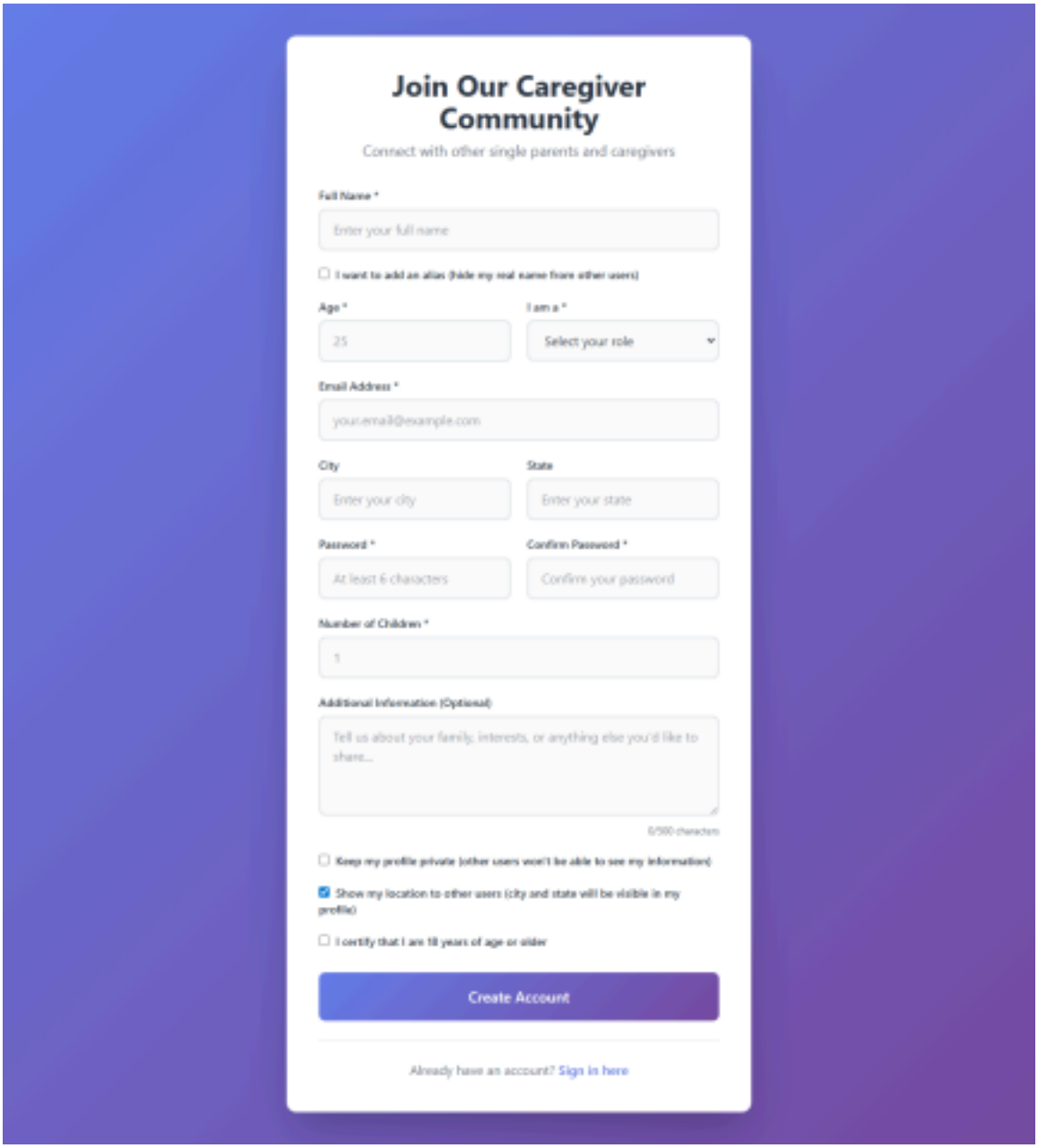


Figure 2: Sign Up Form – top section

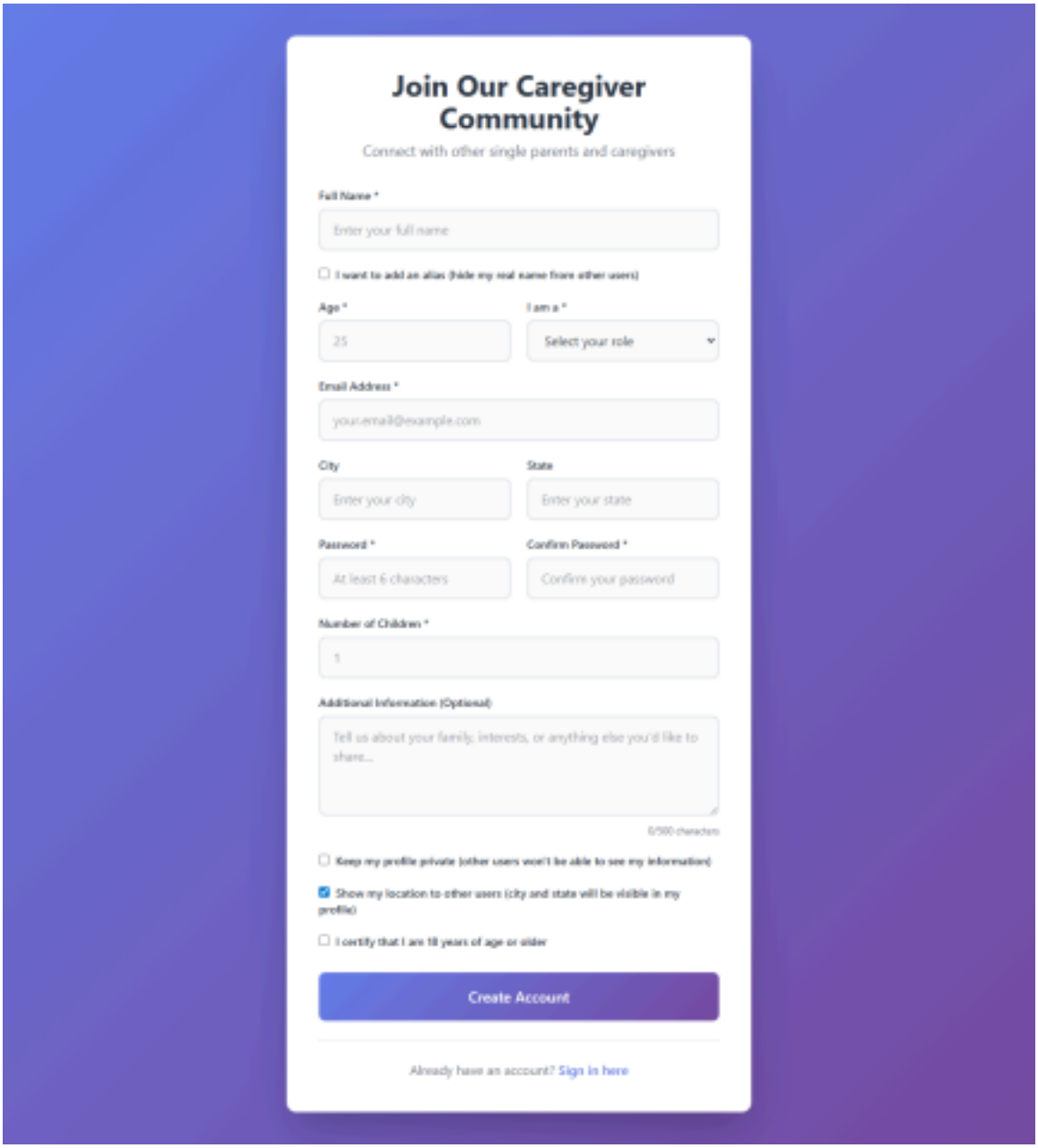


Figure 3: Sign Up Form – bottom section

3.1 Form Fields

Field	Type / Constraints	Required
Full Name	Text, 2-50 characters	Yes
Alias	Text, 2-50 characters. Displayed instead of real name in posts/discussions.	No
Age	Number, must be 18-100	Yes
Caregiver Type	Select: Parent, Guardian, or Grandparent	Yes

Email	Valid email format, must be unique	Yes
Password	Minimum 6 characters	Yes
Confirm Password	Must match the Password field	Yes
Number of Children	Number, 1-20	Yes
Children's Age Groups	One dropdown per child, options: 0-3, 4-6, 7-12, 13-18	Yes
City	Text, max 50 characters	No
State	Text, max 50 characters	No
Additional Information	Textarea, max 500 characters. Free-text background info.	No
Keep my profile private	Checkbox. When checked, profile is hidden from community list.	No
Show my location to other users	Checkbox. When checked, city/state appear on public profile.	No
I am 18 or older	Checkbox certification. Must be checked to submit.	Yes

3.2 Validation and Error Handling

- Missing required fields are highlighted in red with inline error messages.
- Password and Confirm Password must match.
- Age must be between 18 and 100.
- Email must be unique. The server returns an error if the email is already registered. ● Number of children must be between 1 and 20; age-group dropdowns are generated dynamically to match.
- The age certification checkbox must be checked before submission.

3.3 On Success

After successful submission, a confirmation message is displayed. The user is automatically redirected to /login after 2 seconds.

4. Login

URL Path: /login

The login page authenticates existing users. On success, the full user object is stored in the browser's localStorage so the session persists across page refreshes.

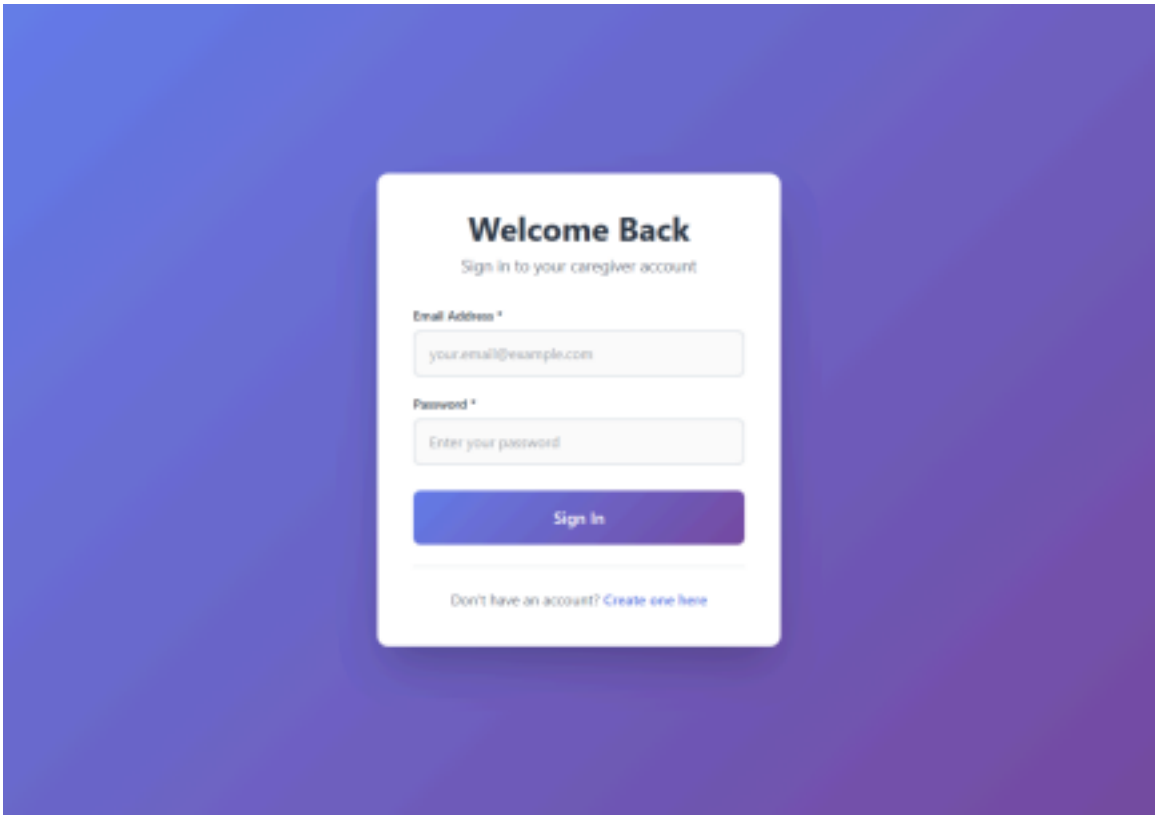


Figure 4: Login Page – empty form

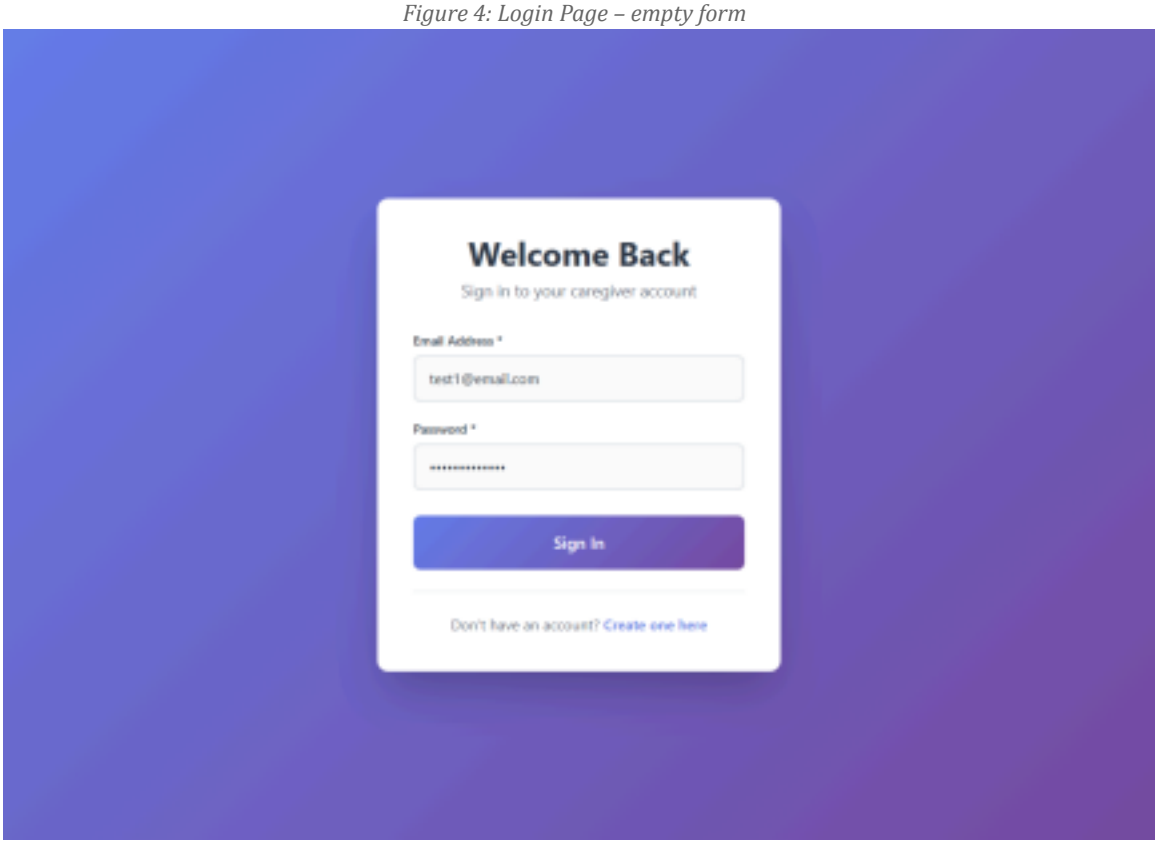


Figure 5: Login Page – credentials entered

4.1 Form Fields

Field	Type / Constraints	Required
Email Address	Valid email format	Yes

Password	Minimum 6 characters	Yes

4.2 Behavior

On successful login, the user's full profile object (excluding password) is saved to localStorage under the key 'user'. The user is then redirected to /home.

On failure, an error message is displayed below the form (e.g., 'Invalid email or password').

4.3 Session Persistence

The application does not use JWT tokens or server-side sessions. The user object stored in localStorage acts as the session. The user's MongoDB _id is sent as the 'user-id' header on every authenticated API request. Logging out clears this localStorage entry and redirects to /login.

5. Home Page – Community Members

URL: /home | Tab: Home

The Home tab is the first view shown after login. It displays a list of other active community members whose profiles are not set to private.

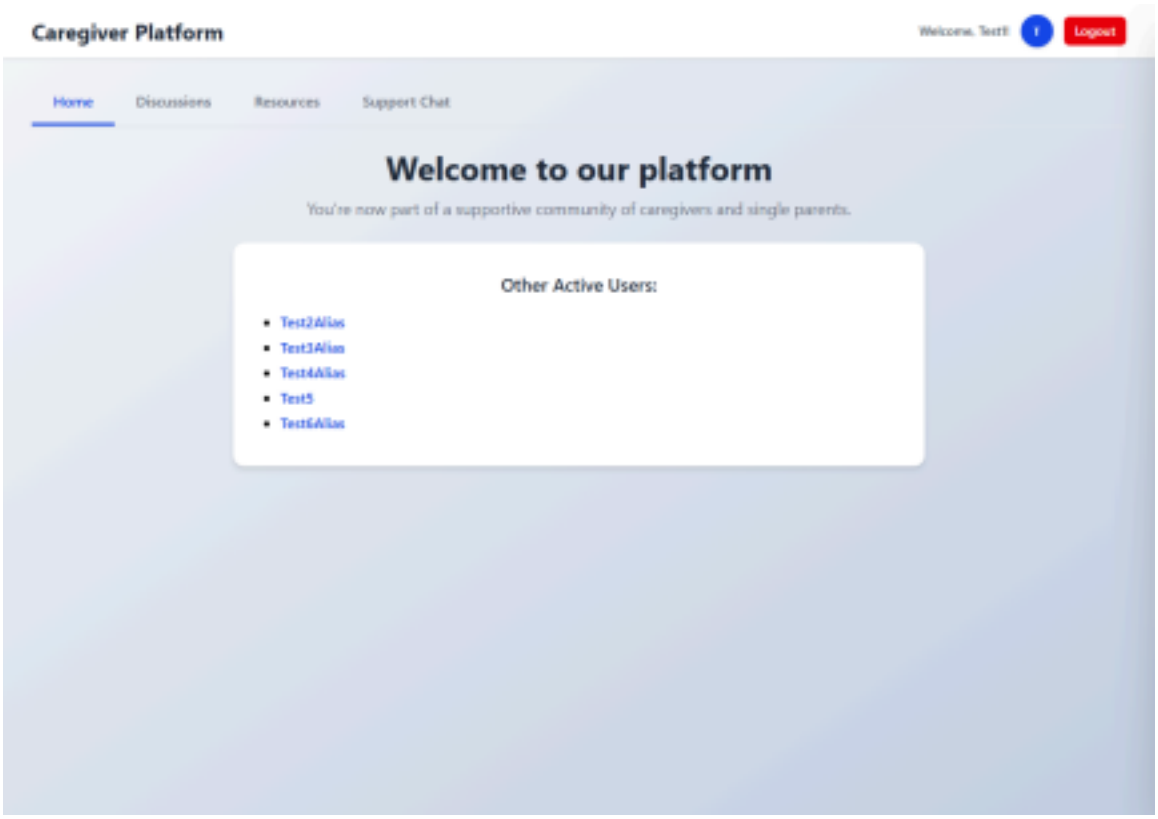


Figure 6: Home Tab – Community Members

5.1 Page Layout

Header / Navigation Bar: Displays 'CaregiverConnect', a greeting with the user's name, an avatar circle showing initials, and a Logout button.

Tab Bar: Four tabs: Home, Discussions, Resources, Support Chat. The active tab is highlighted with a bottom border.

Member List: Cards for each non-private community member, laid out in a responsive grid.

5.2 Member Cards

- Name or Alias (whichever the member has chosen to display)
- Caregiver Type (Parent, Guardian, or Grandparent)
- Number of children and their age groups
- Location (city, state), shown only if the member enabled 'Show my location'
- Member since date

Cards can be expanded to reveal the full profile. Private profiles (users who opted in to privacy) are excluded from this list entirely.

5.3 Header Controls

Avatar Circle: Displays the user's initials. Click to open the Profile panel (see Chapter 6).

Logout Button: Clears the localStorage session and redirects to /login.

6. Profile Management

Accessed via: Avatar button in the top-right header area

The Profile panel slides in from the right side of the screen and lets users update their personal information, privacy settings, and family details without leaving the page.

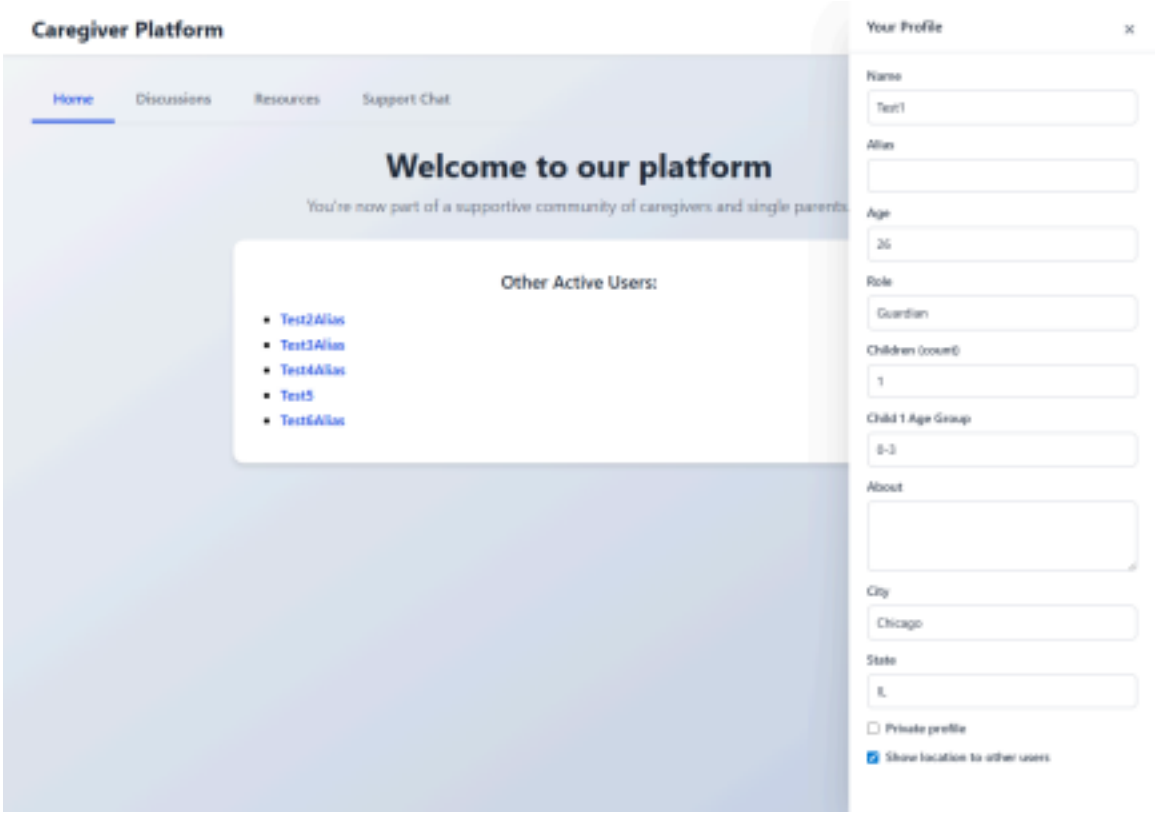


Figure 7: Profile Panel – slide-out

6.1 Editable Fields

Field	Type / Constraints	Required
Full Name	Text, 2-50 characters	Yes

Alias	Text, 2-50 characters	No
Age	Number, 18-100	Yes
Caregiver Type	Select: Parent, Guardian, Grandparent	Yes
Number of Children	Number, updates age-group dropdowns dynamically	Yes
Children's Age Groups	One dropdown per child: 0-3, 4-6, 7-12, 13-18	Yes
City	Text, max 50 characters	No
State	Text, max 50 characters	No
Keep my profile private	Toggle checkbox	No
Show my location to other users	Toggle checkbox	No

6.2 Saving Changes

Click 'Save Changes' to submit the updated profile to the server. The localStorage session data is updated immediately to reflect the changes. The page header will show the updated name or alias right away.

6.3 Privacy Controls

Keep my profile private: When enabled, the user's card is hidden from the Home tab's member list. The user can still access all features normally.

Show my location: When enabled, the user's city and state appear on their community member card. When disabled, location fields are stored but not displayed publicly.

7. Discussions

URL: /home | Tab: Discussions

The Discussions tab is the community forum. Users can browse and participate in topic-based conversations. Topics are created by admins or promoted from user requests.

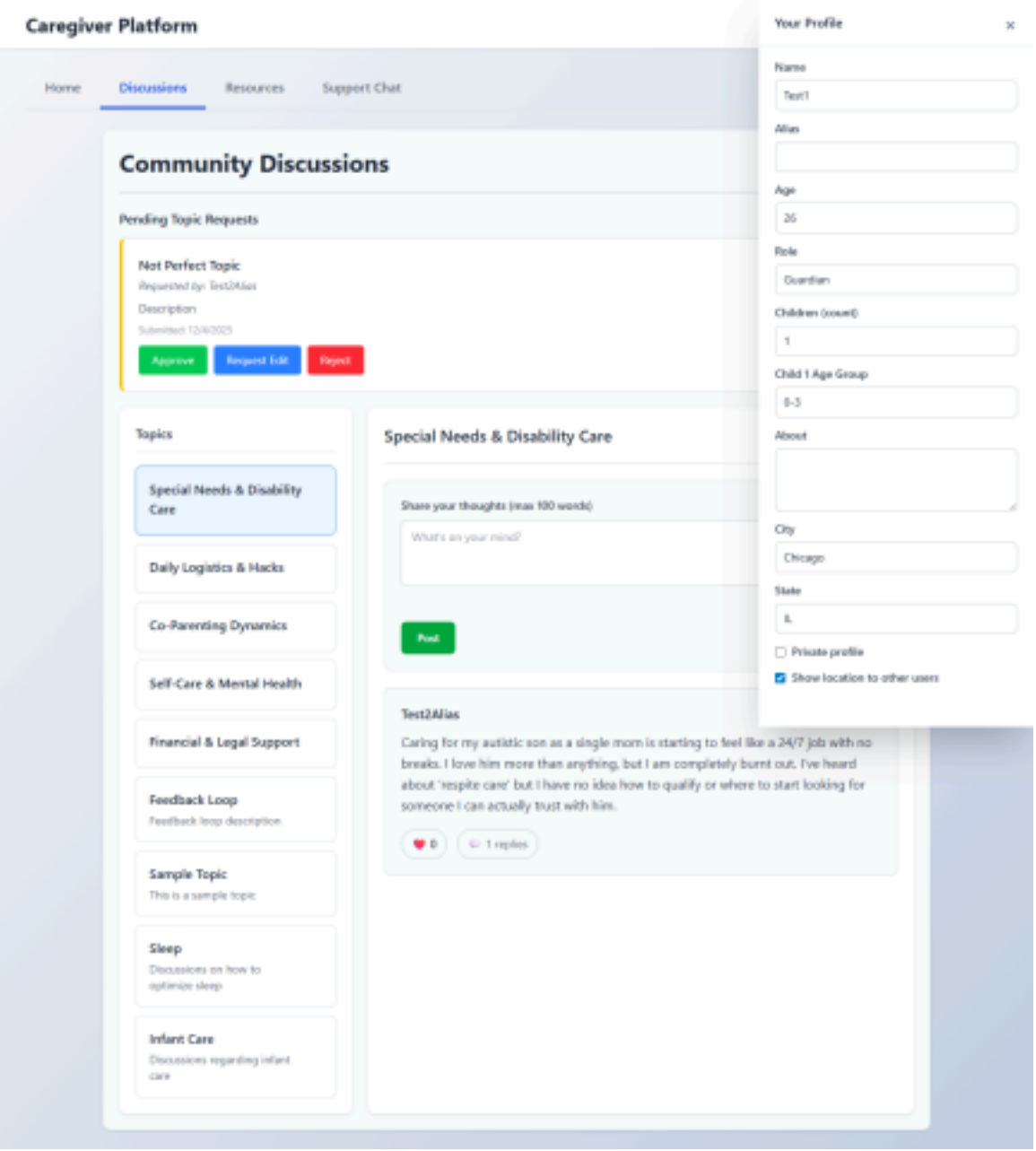


Figure 8: Discussions Tab – overview

7.1 Browsing Topics and Posts

The left sidebar lists all active discussion topics. Clicking a topic loads its posts in the main content area. Posts are paginated (10 per page) and sorted by creation date.

Each post card shows:

- Author name or alias
- Post content (max 100 words)
- Creation date and time
- Like count and a heart button to like the post
- Reply count and a speech bubble button to toggle replies

7.2 Creating Posts and Replies

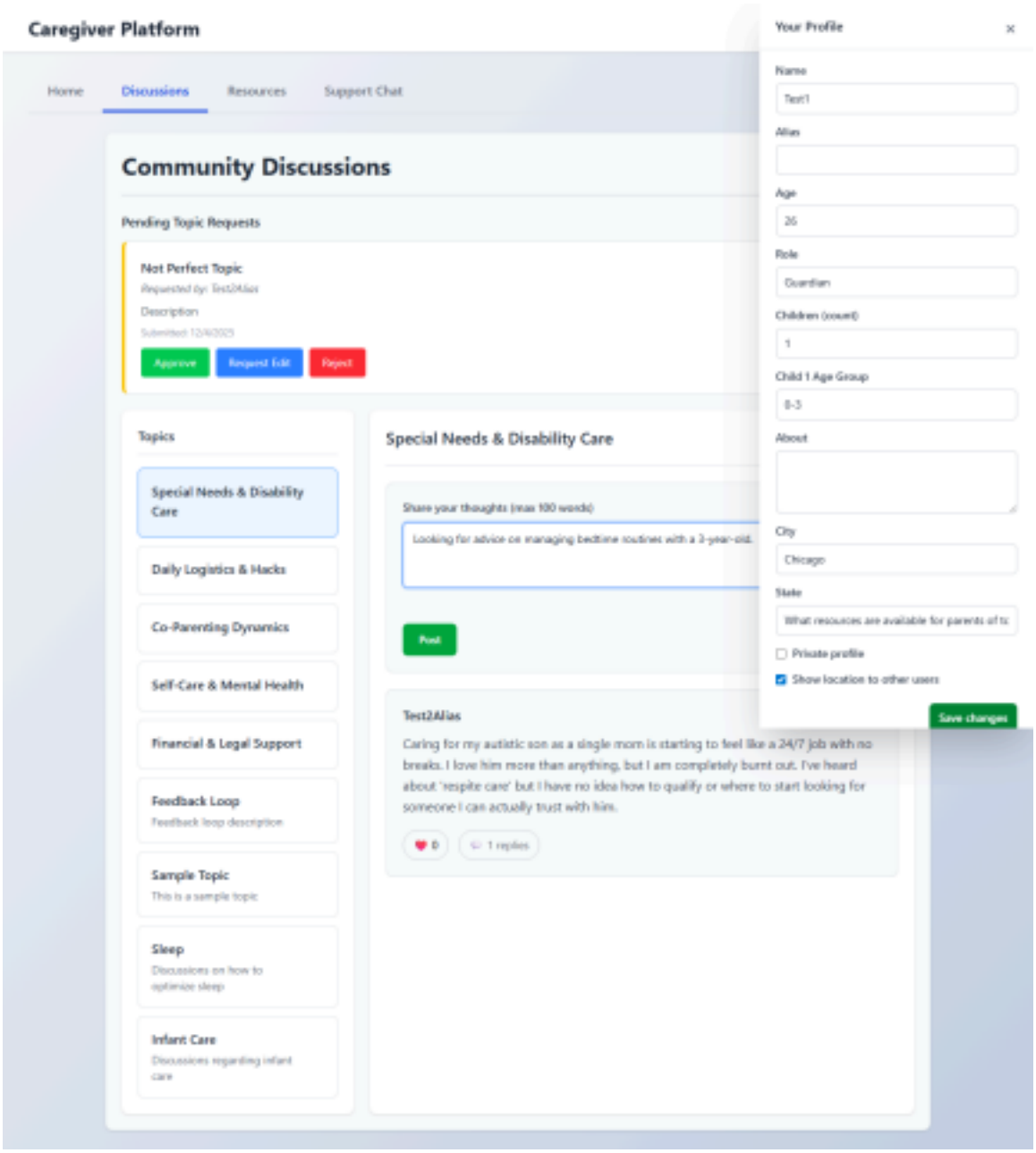


Figure 9: Discussions – composing a post

To create a post: select a topic from the sidebar, type your message in the post text area (a live word counter shows remaining words out of 100), then click 'Post'.

To reply to a post:

1. Click the speech bubble icon on a post to expand the reply section.

2. Type your reply in the text field (max 500 characters).
3. Click 'Reply' to submit.

Both posts and replies support likes. Each like increments the like counter. There is no un-like feature.

7.3 Topic Requests (Users)

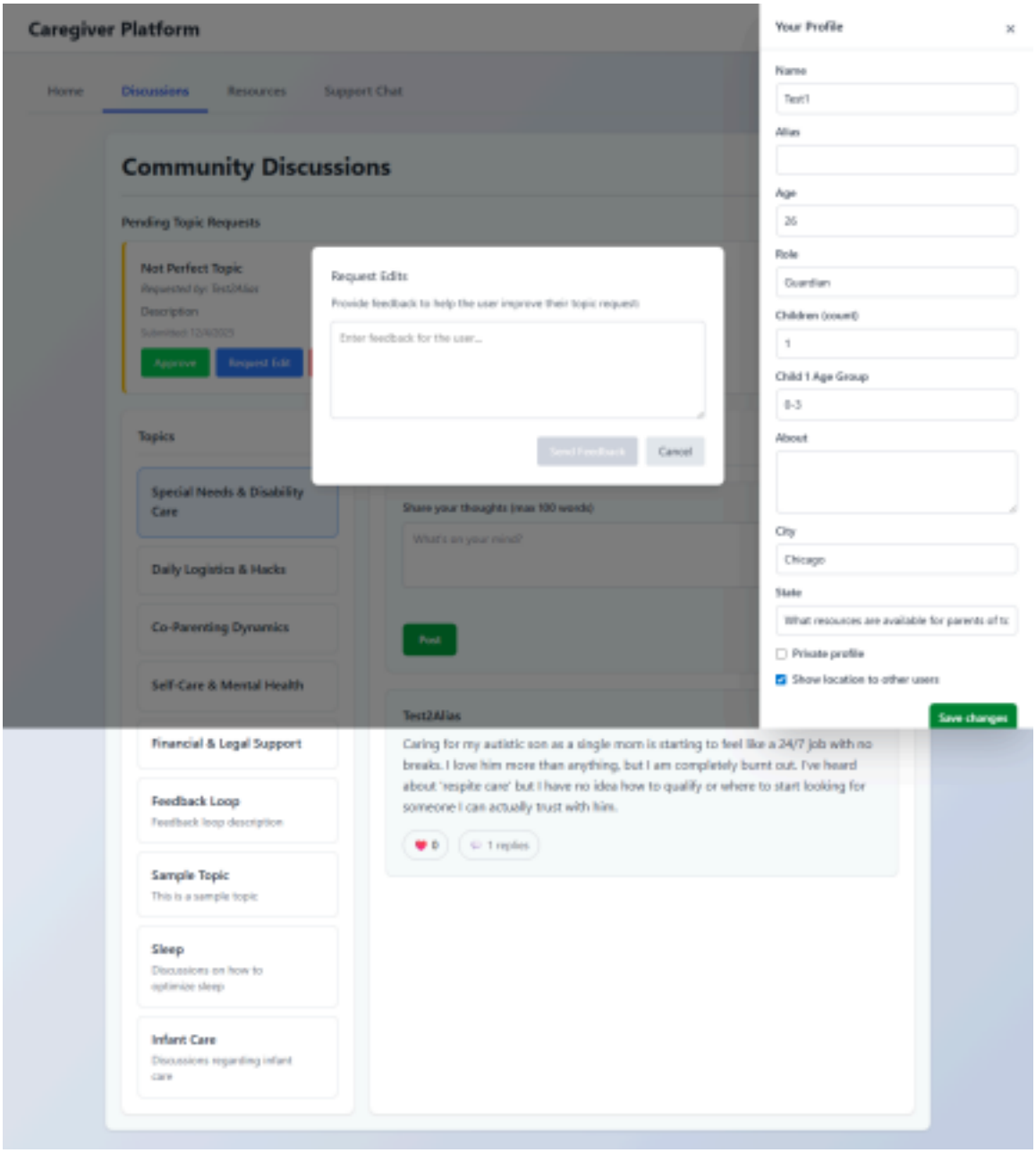


Figure 10: Topic Request Form

If a user wants a topic that does not exist, they can submit a topic request. Click 'Request a Topic' to expand the request form.

Field	**Type / Constraints**	**Required**
Topic Name	Text, max 50 characters	Yes
Description	Text, max 200 characters. Explain why this topic is needed.	No

Request status workflow:

Status	**Meaning**	**Color**

Pending	Submitted and awaiting admin review.	Yellow
Edit Requested	Admin asked for changes. User can edit and resubmit.	Blue
Approved	Admin approved; topic created.	Green
Rejected	Admin rejected. A reason may be provided.	Red

7.4 Admin: Managing Topics and Requests

Admin users see additional controls in the Discussions tab. An admin panel is visible only to accounts with admin privileges (admin: true in the database).

Add Topic: Form with topic name (2-50 chars) and description (max 200 chars). Creates the topic immediately.

Delete Topic: Removes the topic and all its posts permanently.

View Pending Requests: A section listing all Pending and Edit_Requested topic requests from users.

Approve Request: Creates the topic from the request data and marks the request Approved.

Request Edits: Admin provides feedback text (max 500 chars); status changes to Edit_Requested.

Reject Request: Admin may provide a rejection reason (max 500 chars); status changes to Rejected.

8. Resources

URL: /home | Tab: Resources

The Resources tab provides a curated library of external links vetted and added by admins. Each resource is a link to an external website with a title and description.

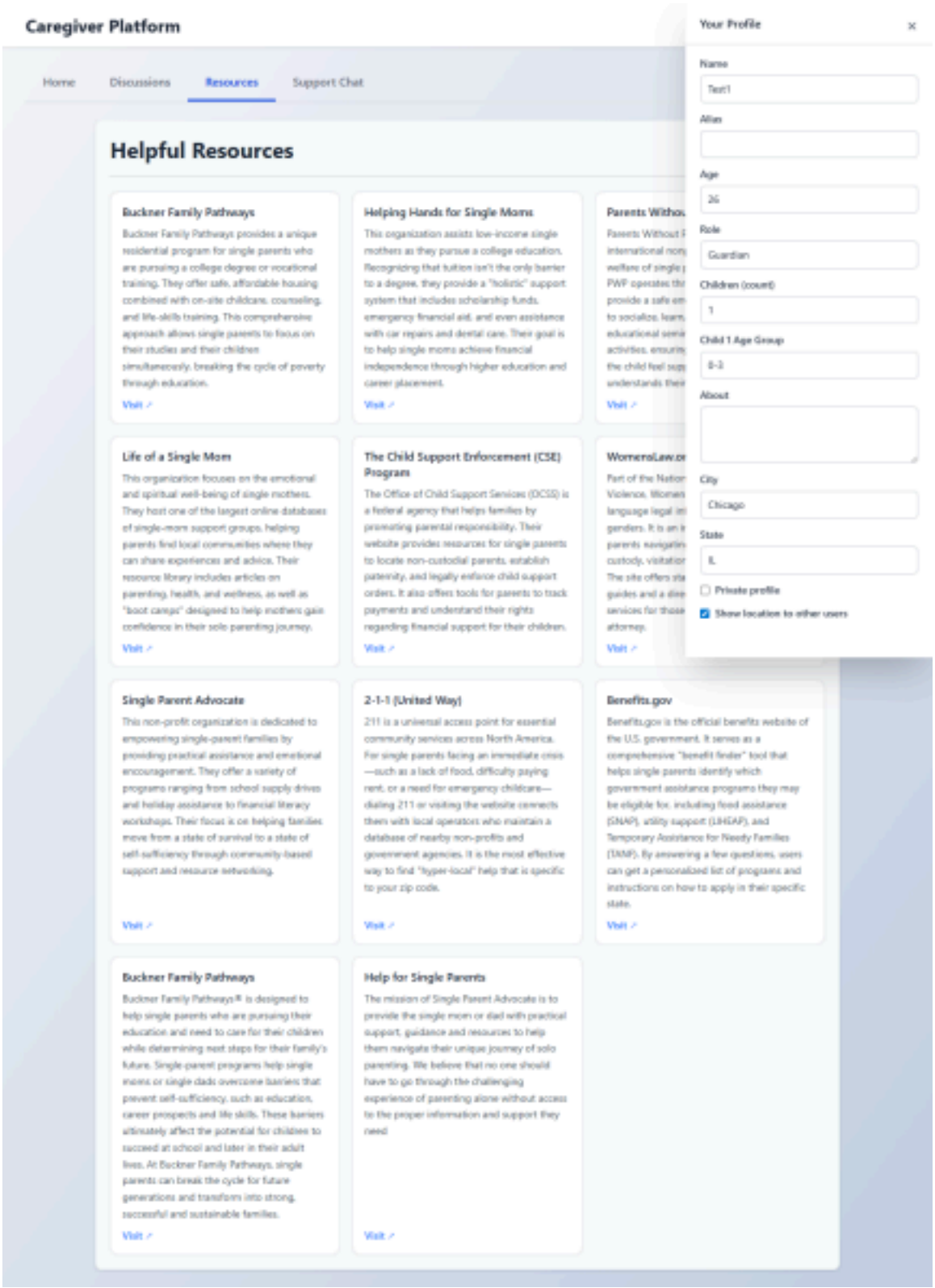


Figure 11: Resources Tab – resource grid

8.1 Resource Cards

- Title – the name of the resource
- Description – a brief summary (max 100 words)
- 'Visit' link – opens the resource URL in a new browser tab

The grid is responsive: 1 column on mobile, 2 columns on tablets, 3 columns on desktop.

8.2 Admin: Adding Resources

Admins see an 'Add Resource' button above the grid. Clicking it reveals a form:

Field	Type / Constraints	Required
Title	Text, 2-120 characters	Yes
URL	Must start with http:// or https://	Yes
Description	Text, max 100 words (approx. 1200 characters)	Yes

After saving, the resource is stored in the database and an embedding is automatically generated for use in the AI Support Chat's retrieval system.

9. Support Chat (AI-Powered)

URL: /home | Tab: Support Chat

The Support Chat tab provides a 24/7 AI assistant trained on the community's posts and curated resources. It uses a Retrieval-Augmented Generation (RAG) pipeline to answer questions with context drawn directly from the community.

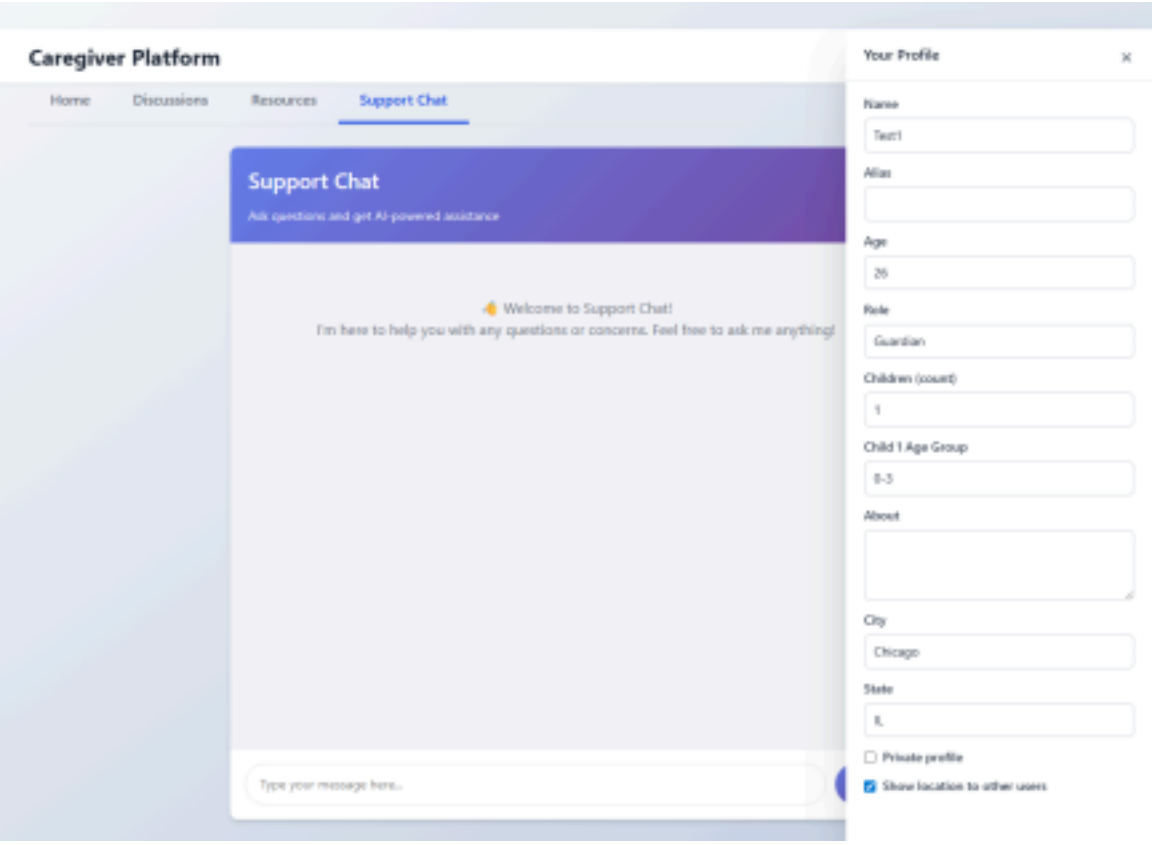


Figure 12: Support Chat – empty state

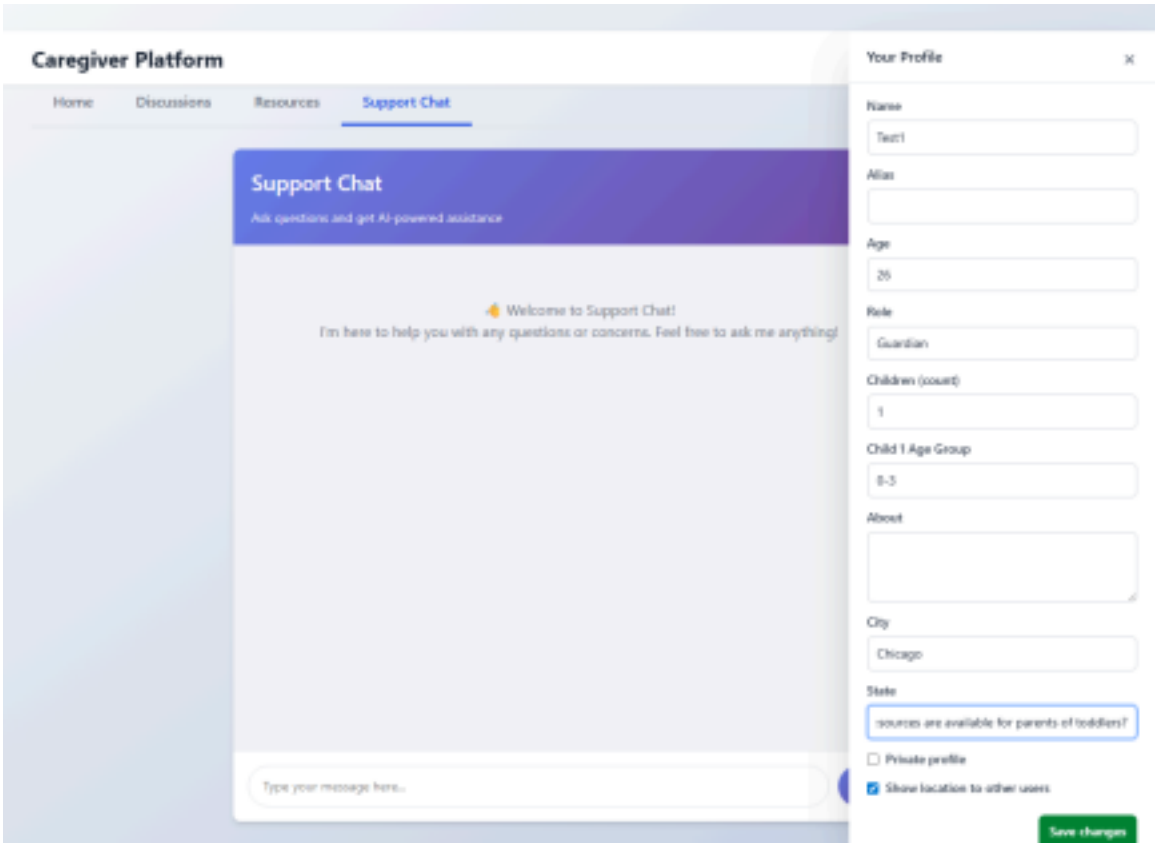


Figure 13: Support Chat – message typed

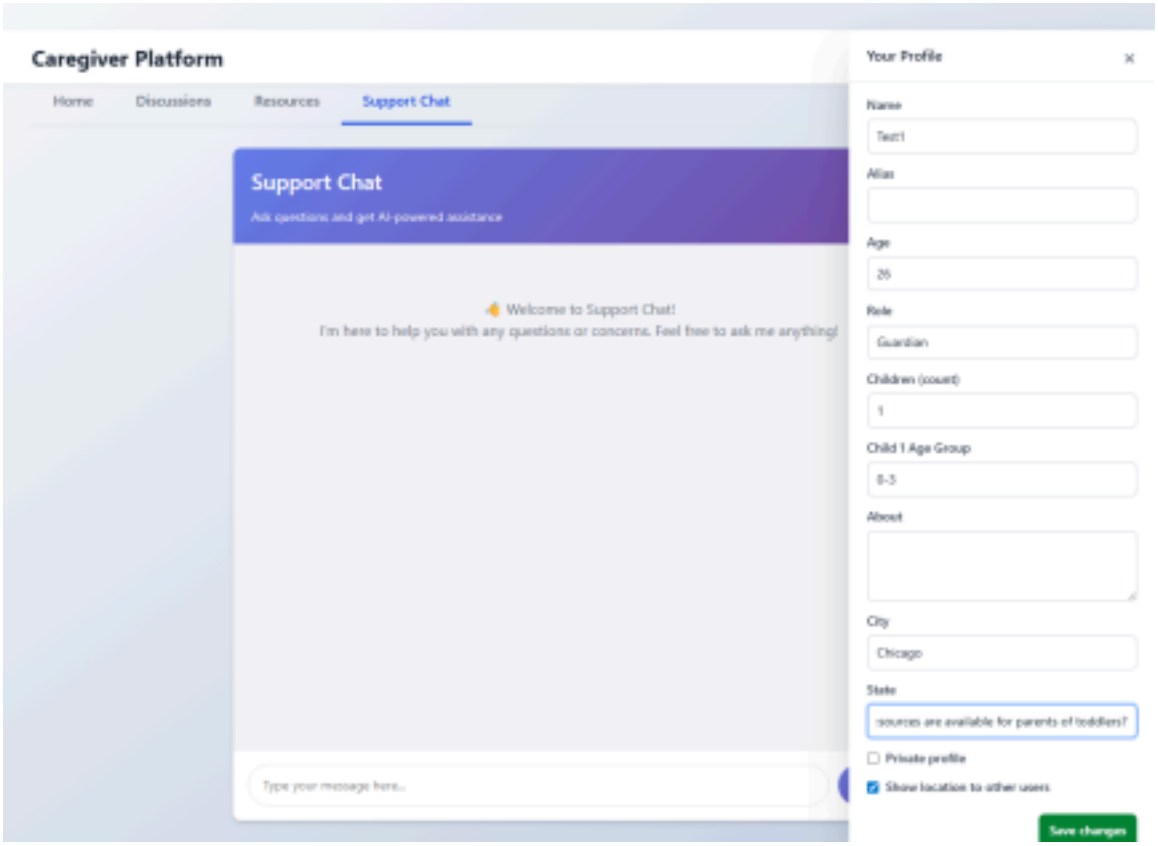


Figure 14: Support Chat – waiting for response (loading animation)

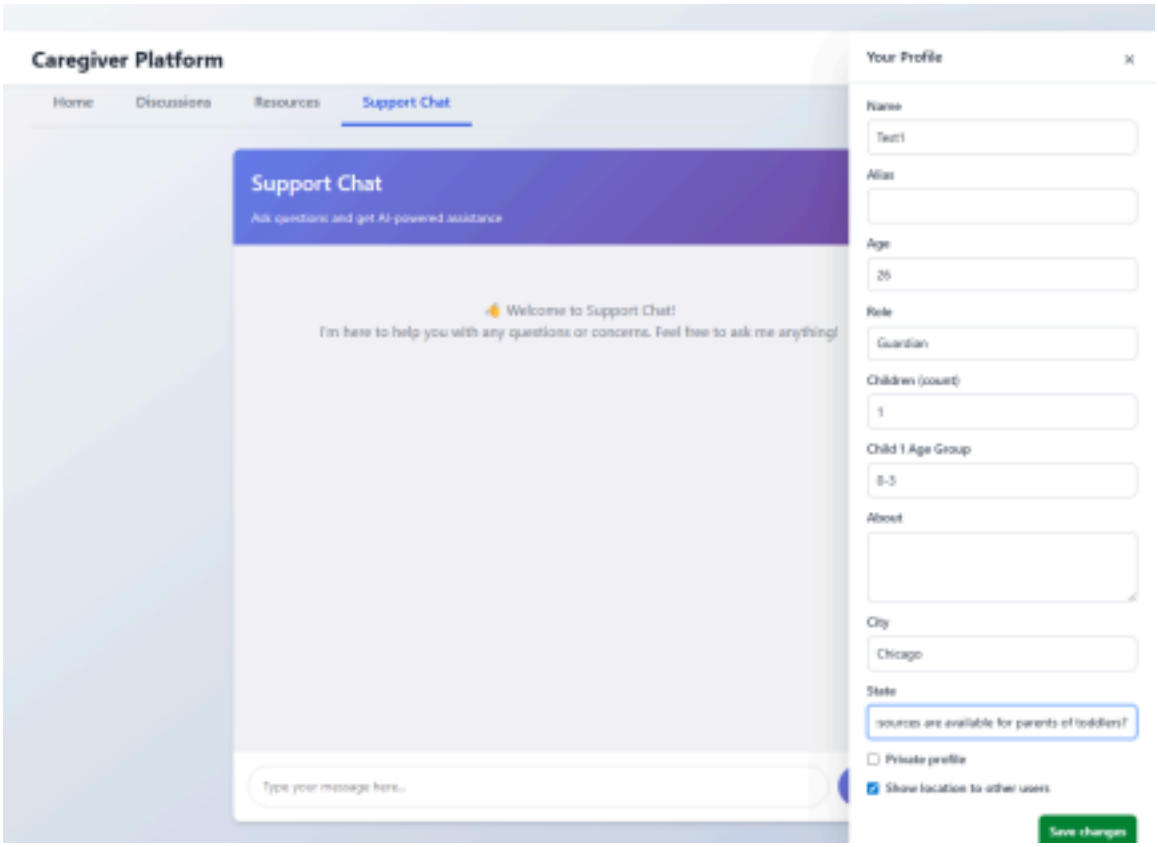


Figure 15: Support Chat – AI response with references

9.1 How to Use

1. Type your question in the text field at the bottom of the chat.

2. Press Enter or click the Send button.

3. A loading animation (three bouncing dots) appears while the AI processes your question. 4.

The AI's response appears as a chat bubble on the left side, rendered as formatted markdown.

5. If the AI found relevant community posts or resources, they appear below the response as reference cards.

9.2 Chat Interface Elements

User messages: Blue pill-shaped bubbles on the right side of the screen.

AI messages: White bubbles on the left side, supporting markdown formatting (bold, italics, lists, tables, code blocks).

Loading indicator: Three animated dots indicating the AI is generating a response.

Error messages: Red bubbles with a border if the request fails.

Timestamps: Each message shows the time it was sent.

Clear Chat button: Clears all messages. Chat history is only stored in sessionStorage and not persisted to the server.

Reference Cards – Posts: Green-accented cards showing community post content, author, topic, and date used as AI context.

Reference Cards – Resources: Blue-accented cards showing the resource title (clickable link), description excerpt, and a link icon.

9.3 How the AI Works Internally

The Support Chat uses a Retrieval-Augmented Generation (RAG) pipeline. The AI first searches the community's content for relevant information, then uses that as context to generate its response. The pipeline has seven steps:

Step 1: Query Embedding: The user's message is converted into a 384-dimensional vector using the Xenova/all-MiniLM-L6-v2 sentence transformer model, capturing the semantic meaning of the question.

Step 2: Vector Search: The query embedding is compared against embeddings stored in MongoDB Atlas Vector Search. Two indexes are searched in parallel, 'posts_index' (community posts) and 'resources_index' (curated resources). The top 10 candidates from each collection are retrieved using cosine similarity, with a minimum score threshold of 0.6.

Step 3: Cross-Encoder Reranking: The 10 candidates from each collection are reranked using the Xenova/bge-reranker-base cross-encoder model, which scores each query-document pair for relevance. Documents below a sigmoid threshold of 0.5 are filtered out. The top 3 posts and top 3 resources pass through to the LLM.

Step 4: History Compaction: Previous messages in the session are summarised into 3-5 bullet points using Groq's llama-3.1-8b-instant model. This keeps conversation continuity without sending the full history each time. Requires GROQ_API_KEY; skipped if not configured.

Step 5: Prompt Construction: A prompt is assembled from the retrieved post and resource context, the conversation summary, and the user's current question. The AI is instructed to answer only using the provided community context and to be empathetic and supportive.
Step 6: LLM Generation: The prompt is sent to Google Gemini 2.5 Flash, which generates a markdown-formatted response grounded in the community context.

Step 7: Response and References: The API returns the AI's answer along with metadata about the posts and resources used as context. The frontend renders the answer as markdown and displays reference cards below it.

9.4 Embedding Generation (Automatic)

Every time a community post or resource is saved, an embedding is automatically generated and stored alongside it via a Mongoose pre-save hook. New content becomes searchable by the AI chat

without manual indexing. If embedding generation fails (for example, on cold start), the document still saves; it simply won't be retrieved by the AI until an embedding is generated successfully.

9.5 Chat History and Session Storage

Chat history is stored in the browser's sessionStorage. This persists if you refresh the page within the same browser session, but is cleared when you close the tab or browser. Clicking 'Clear Chat' also wipes the stored history immediately.

9.6 Example Questions

The examples below show real questions the Support Chat is designed to help with. For each one, the AI searches community posts and curated resources before answering, and displays the matching references below its response.

Example 1: Community and isolation

Question: Where can I find a community or a local chapter of other single parents to hang out with so I don't feel so isolated?

The AI looks for posts and resources about peer support, local groups, and social connection for single parents.

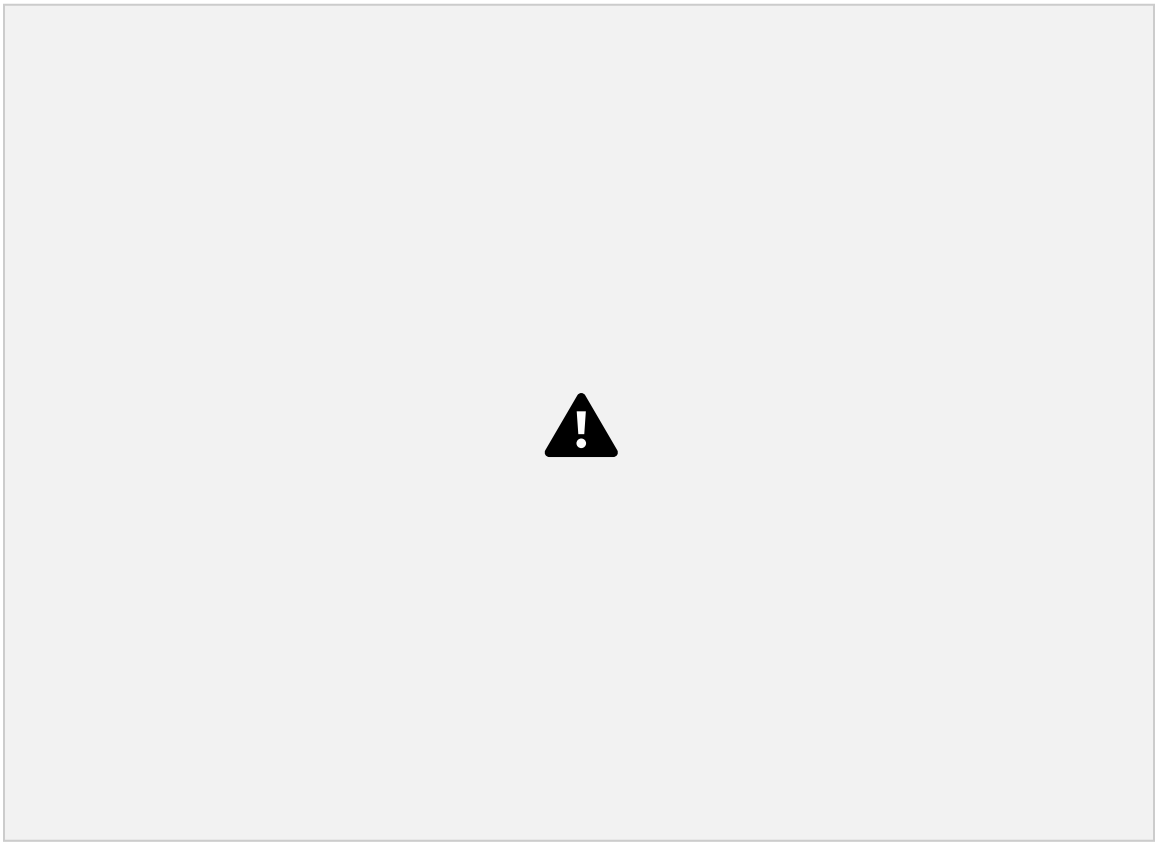


Figure 16: Support Chat – community connection example

Example 2: Legal rights: custody and visitation

Question: I need to understand my legal rights regarding child custody and visitation, but I can't afford a lawyer. Is there a site that explains these laws simply?

The AI retrieves posts and resources on legal aid, free legal resources, and custody processes, and answers in plain language.

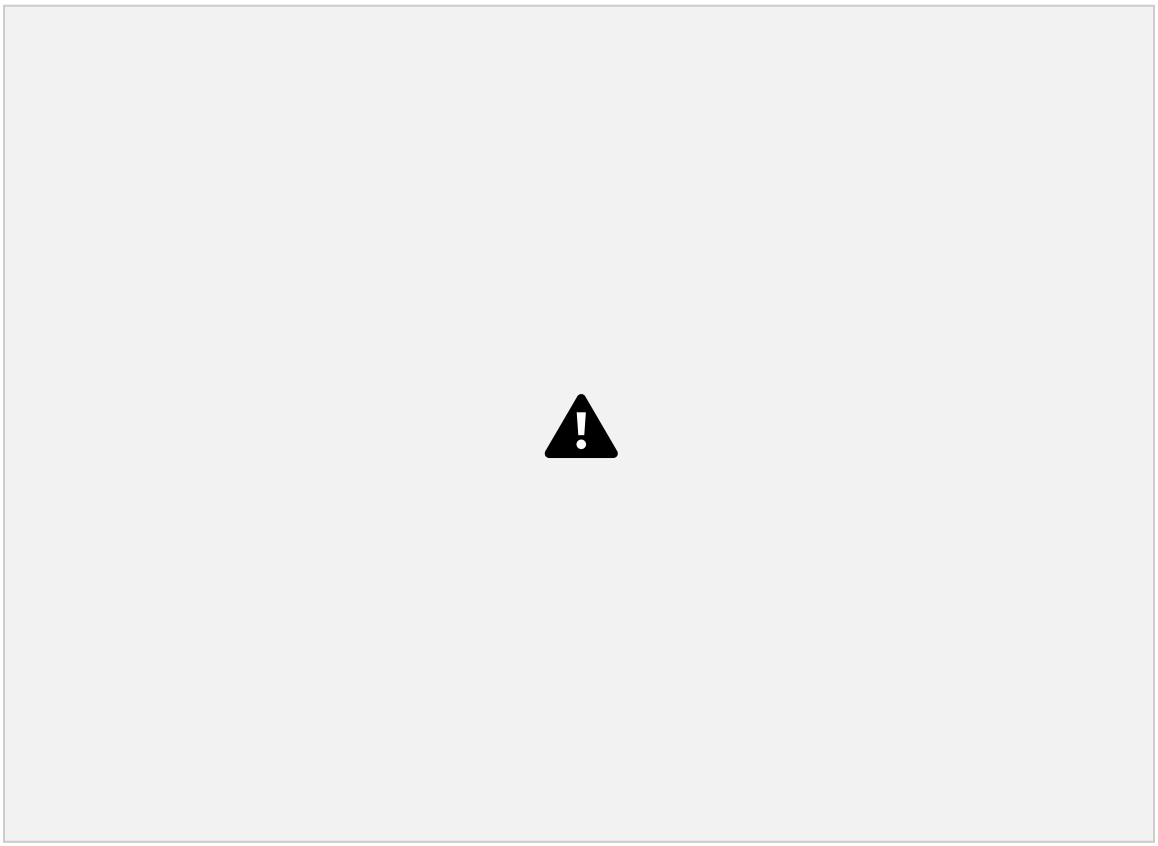


Figure 17: Support Chat – legal rights example

Example 3: Financial aid: scholarships for single parents

Question: I'm a single mom trying to finish my degree but I'm struggling with car repairs and childcare costs. Are there scholarships that cover more than just tuition?

The AI looks for posts and resources on financial assistance, emergency funds, and scholarships that cover living costs.

Example 4: Housing: residential programs for single parents

Question: Is there a residential program or a safe place to live that helps single parents finish school while providing childcare on-site?

The AI checks for posts and resources on transitional housing and supportive housing programs.

Example 5: Child support enforcement

Question: I need help enforcing a child support order. What government agency handles locating a non-custodial parent and making sure they pay?
The AI surfaces posts from members who have navigated enforcement, plus resource links to government and legal aid agencies.

Example 6: Meal prep for single parents

Question: I'm struggling with the 'what's for dinner' chaos every night. Does anyone have a meal prep routine that actually works for a single parent schedule?

The AI searches community posts for meal prep tips shared by other single parents and links to relevant resources.

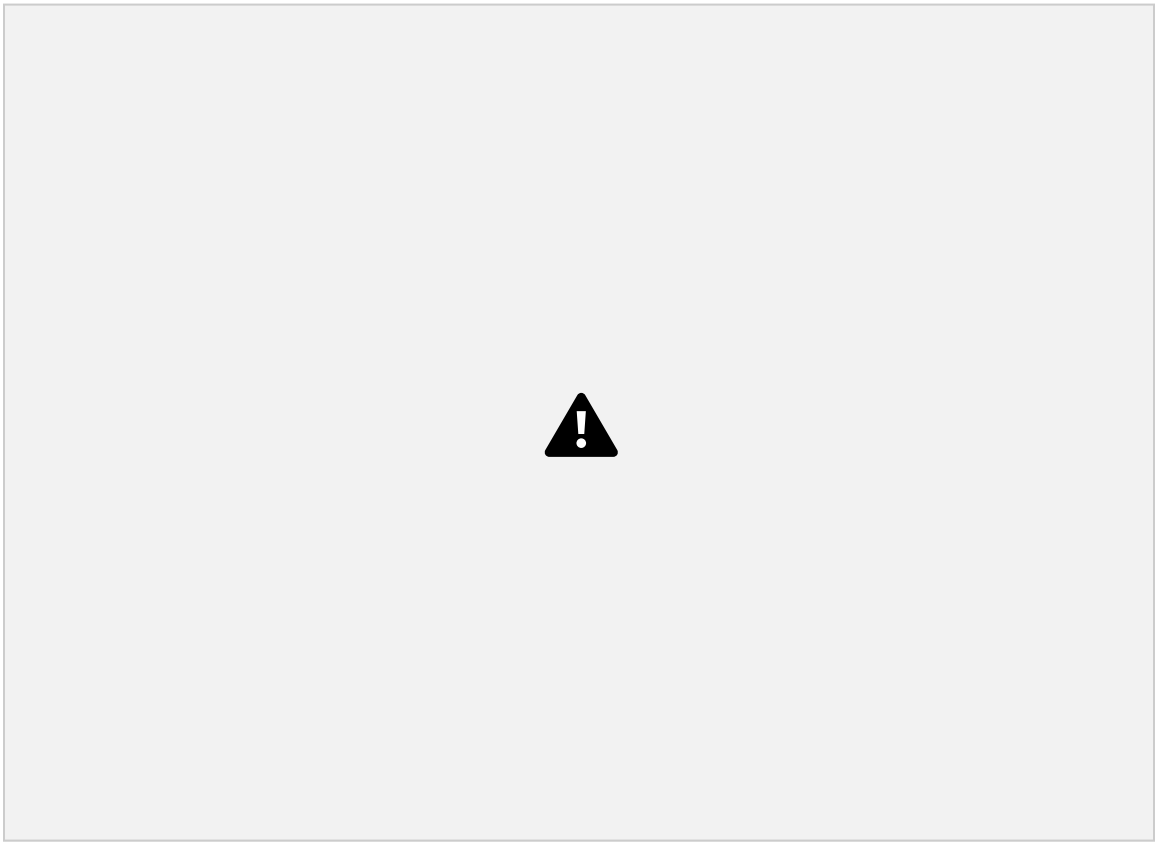


Figure 18: Support Chat – meal prep example

Example 7: Free activities for kids on a tight budget

Question: I want to take my kids out this weekend but I'm on a really tight budget. Are there any 'hidden gem' free activities you'd recommend?

The AI surfaces posts where community members share free local activities and budget-friendly ideas.

10. Technical Reference

10.1 Architecture Overview

CaregiverConnect is a full-stack web application. The frontend (React/Vite) runs on port 5173 during development. The backend (Express) runs on port 5000. All API calls are prefixed with /api/ and target http://localhost:5000, which is hardcoded with no Vite proxy. A production deployment would need a reverse proxy or environment-based configuration.

Layer	Path	Stack
Frontend	src/	React 19, Vite 6, Tailwind CSS 4, React Router v6

Backend	server/	Node.js, Express 5, MongoDB/Mongoose 8
Utilities	server/utils/	embeddings.js, rag.js, reranker.js, chatMemory.js
Models	server/models/	User, Topic, Post, Resource, TopicRequest
Routes	server/routes/	auth.js, discussions.js, resources.js, chat.js
Config	server/config/	rag.config.js, RAG pipeline tuning parameters

10.2 AI / RAG System

Models used:

Model	**Role**	**Provider**	**Notes**
all-MiniLM-L6-v2	Sentence embedding	Xenova/HuggingFace	384-dim vectors, runs in Node.js via @xenova/transformer s
bge-reranker-base	Cross-encoder reranking	Xenova/HuggingFace	Scores query-document pairs for relevance, runs in Node.js
gemini-2.5-flash	Response generation	Google Gemini API	LLM that synthesizes community context into answers

llama-3.1-8b-instant History compaction Groq API (optional) Summarises chat history to save tokens on long conversations

RAG configuration parameters (server/config/rag.config.js):

Parameter	**Default**	**Description**
vectorSearch.candidateLimit	10	Candidates retrieved from vector search before reranking

vectorSearch.minScore	0.6	Minimum cosine similarity to include a candidate
reranker.model	Xenova/bge-reranker-base	Cross-encoder model identifier
reranker.threshold	0.5	Minimum sigmoid score for a document to pass reranking
llm.postsLimit	3	Max posts passed as context to the LLM
llm.resourcesLimit	3	Max resources passed as context to the LLM

MongoDB Atlas Vector Search Setup

Two Atlas Search indexes must be created manually in MongoDB Atlas before the RAG system functions. Both are vector indexes on the 'embedding' field (384 dimensions, cosine similarity):

posts_index: on the posts collection, 'embedding' field

resources_index: on the resources collection, 'embedding' field

10.3 Data Models

User

- name (String, 2-50 chars)
- alias (String, optional, 2-50 chars, shown instead of real name)
- age (Number, 18-100)
- caregiverType (String: parent | guardian | grandparent)
- email (String, unique)
- password (String, bcrypt hashed)
- familyInfo.numberOfKids (Number, 1-20)
- familyInfo.kidsAgeGroups (Array of: 0-3 | 4-6 | 7-12 | 13-18)
- familyInfo.additionalInfo (String, max 500)
- isPrivate (Boolean, default false)
- location.city / location.state (String, max 50 each)
- showLocation (Boolean, default true)
- admin (Boolean, default false)
- lastLogin (Date)
- createdAt (Date)

Topic

- name (String, unique, 2-50 chars)
- description (String, max 200)

- isActive (Boolean, default true)
- createdBy (ObjectId ref: User)
- createdAt / updatedAt (Date)

Post

- content (String, max 100 words, validated by Mongoose)
- author (ObjectId ref: User)
- authorName (String, denormalized at write time)
- topic (ObjectId ref: Topic)
- topicName (String, indexed)
- likeCount (Number, default 0)
- embedding (Array<Number>, 384-dim, optional)
- replies: content (max 500 chars), author, authorName, createdAt, likeCount
- createdAt (Date)

Resource

- title (String, 2-120 chars)
- url (String, http/https)
- description (String, max approx. 100 words)
- createdBy (ObjectId ref: User)
- isActive (Boolean, default true)
- embedding (Array<Number>, 384-dim, optional)
- createdAt / updatedAt (Date, auto)

TopicRequest

- name (String, max 50)
- description (String, max 200)
- status (String: Pending | Approved | Rejected | Edit_Requested)
- userId (ObjectId ref: User, indexed)
- reviewedBy (ObjectId ref: User)
- reviewedAt (Date)
- feedbackText (String, max 500)
- rejectionReason (String, max 500)
- createdAt / updatedAt (Date)

10.4 REST API Reference

Authentication (/api/auth)

Method	Endpoint	Description

POST	/api/auth/signup	Create account. Body: name, alias, age, caregiverType, email, password, familyInfo, location, isPrivate, showLocation
POST	/api/auth/login	Login. Body: email, password. Returns full user object.
GET	/api/auth/users	List non-private users. Query: currentUserId
GET	/api/auth/users/:id	Get user by MongoDB _id
PUT	/api/auth/users/:id	Update profile. Header: user-id

Discussions (/api/discussions)

Method	**Endpoint**	**Description**
GET	/api/discussions/topics	List all active topics
POST	/api/discussions/topics	Create topic (admin only). Body: name, description
DELETE	/api/discussions/topics/:id	Delete topic and all its posts (admin only)
GET	/api/discussions/topics/:topic Id/posts	Get posts for topic. Query: page
POST	/api/discussions/topics/:topic Id/posts	Create post. Header: user-id. Body: content
POST	/api/discussions/posts/:postI d/like	Increment post like count
POST	/api/discussions/posts/:postI d/replies	Add reply. Header: user-id. Body: content
POST	/api/discussions/posts/:postI d/replies/:replyId/like	Increment reply like count
POST	/api/discussions/topic-requ es ts	Submit topic request. Header: user-id

GET PUT GET POST POST	/api/discussions/topic-requ es ts/my-requests /api/discussions/topic-requ es ts/:id /api/discussions/topic-requ es ts/pending /api/discussions/topic-requ es ts/:id/approve /api/discussions/topic-requ es ts/:id/reject	User's own requests. Header: user-id Edit own request (only when Edit_Requested) Admin: list pending/edit_requested requests Admin: approve request Admin: reject. Body: reason
POST	/api/discussions/topic-requ es ts/:id/request-edit	Admin: request edits. Body: feedbackText

Resources (/api/resources)

Method	Endpoint	Description
GET	/api/resources	List all active resources, sorted newest first
POST	/api/resources	Create resource (admin only). Header: user-id. Body: title, url, description

Chat (/api/chat)

Method	Endpoint	Description
POST	/api/chat	RAG chat. Body: { message: string, history: [{role, content}] }. Returns: { success, response, references: { posts, resources } }

10.5 Environment Variables

Create a .env file in the project root. Never commit this file to version control.

Variable	Status	Purpose
MONGODB_URI	Required	MongoDB Atlas connection string
PORT	Optional (default: 5000)	Port for the Express server
GEMINI_API_KEY	Required for AI chat	Google Gemini API key

GROQ_API_KEY	Optional	Groq API key for chat history compaction. Skipped if not set.

10.6 Running the Application

Command	Description
npm install	Install all dependencies
npm run dev:full	Start both frontend (port 5173) and backend (port 5000) concurrently
npm run dev npm run server	Frontend only Backend only
npm run build	Build frontend for production
npm run lint	Run ESLint on the codebase

10.7 Authorization Model

The application uses a simple header-based authorization model:

● Every authenticated request includes the 'user-id' HTTP header set to the user's MongoDB _id.
● requireAuth middleware checks that this header is present (does not validate or verify the user).
● requireAdmin middleware does a database lookup and verifies user.admin === true. ● There are no JWT tokens or server-side sessions. The user's full profile is stored in localStorage. ● Private profiles (isPrivate: true) are excluded from the public user listing but users retain full access.

11. Deployment Strategy

CaregiverConnect is deployed as two independently-hosted services plus a managed database, all on free tiers.

Live Environment

Piece	Platform	URL
Frontend	Vercel	https://caregiver-platform-alpha.vercel.app/
Backend	Render	https://caregiver-platform-5m1z.onrender.com
Database	MongoDB Atlas	—

11.1 Why a split deployment

The frontend (static Vite build) and backend (long-running Express/Mongoose server) have different hosting needs, so they're deployed separately rather than as one service:

- **Vercel** builds and serves the React app as static assets, auto-rebuilding on every push.
- **Render** runs the Express server as a persistent Node process (required — it holds a live MongoDB connection and runs the embedding/reranking models in-process).
- **MongoDB Atlas** free M0 cluster hosts the database; both posts_index and resources_index vector search indexes (see 10.2) are configured there.

Because frontend and backend are separate origins, the frontend can no longer call a hardcoded http://localhost:5000 as described in 10.1 — see 11.2.

11.2 Environment-based API configuration

src/config.js centralizes the API base URL instead of hardcoding it per-component:

```
const raw = import.meta.env.VITE_API_URL || 'http://localhost:5000';
export const API_BASE = raw.replace(/\/+$/, '');
```

- **Local dev:** VITE_API_URL is unset, so API_BASE falls back to http://localhost:5000 — local workflow (10.6) is unchanged.
- **Production:** VITE_API_URL is set in Vercel's project environment variables to the Render backend URL. Since Vite inlines import.meta.env.* at build time, changing this value requires a new build, not just a redeploy of an existing one (see 11.4).

The backend mirrors this pattern for CORS: CORS_ORIGINS (comma-separated allowlist) restricts which browser origins may call the API in production. Left unset, all origins are allowed — the permissive default used locally.

11.3 Environment variables by platform

Variable	Set on	Purpose
MONGODB_URI	Render	Atlas connection string
GEMINI_API_KEY	Render	AI chat generation
GROQ_API_KEY	Render	Chat history compaction (optional)
CORS_ORIGINS	Render	Allowlist of browser origins (the Vercel URL)
NODE_ENV = production	Render	Suppresses verbose chat prompt/response logging
VITE_API_URL	Vercel	Backend URL, inlined into the frontend build

A GET /api/health endpoint ({status, db}) was added to the backend so Render's health checks (and anyone debugging a deploy) can confirm the server is up and the database is connected without touching the ML pipeline.

11.4 Deployment gotchas encountered

A few non-obvious issues came up getting from "code works locally" to "site works for real users," worth documenting so they aren't rediscovered:

- **"Redeploy" rebuilds a pinned commit, not the branch tip.** Both Vercel's and Render's "Redeploy" actions rebuild the exact commit of the deployment you're redeploying from — they do not pull the latest commit on the connected branch. A code change that doesn't show up after redeploying usually means this, not a bug in the code. Fix: trigger a fresh deploy from the branch (e.g. Render's "Deploy latest commit," or a new push).
- **CORS origin mismatch returns 500, not a clear CORS error.** If CORS_ORIGINS is set but doesn't exactly match the frontend's real origin (trailing slash, wrong subdomain, a placeholder value left in by mistake), every browser request fails with a 500 and the browser reports a generic network error — worth checking first when login/signup suddenly "can't connect."
- **Hosting providers may be connected to a fork, not the canonical repo.** In this project, both Vercel and Render are connected to a personal fork rather than the upstream project repository. Forks don't auto-sync with upstream, so a push to the "main" project repo does not deploy until the fork is manually synced (or pushed to directly). Confirm which repository a hosting provider is actually watching (Project Settings → Git) before assuming

a push will deploy.

11.5 Seed / placeholder data

Ahead of real content from the content/HR team, server/seed.js populates the Discussions and Resources tabs with realistic placeholder content so the app isn't empty for early users or demos:

- 5 user accounts (1 admin — "Community Team" — plus 4 caregiver personas)
- 8 discussion topics with 16 seeded posts (several with replies)
- 10 real external resources from reputable organizations (AAP, CDC-adjacent SAMHSA, Postpartum Support International, Child Care Aware of America, etc.)

The script is idempotent — safe to re-run, it skips anything that already exists by email/name/title — and is run manually (npm run seed with MONGODB_URI set) rather than automatically, since it writes real database records.

11.6 Post/reply moderation

Building on the existing topic-request workflow (7.3–7.4), posts and replies now carry their own lightweight moderation state: content from non-admin users is created with status: 'Pending' and is hidden from the public GET /topics/:id/posts response (and from nested replies) until an admin approves it. Admin-authored content is auto-approved. Admins can approve or delete any pending item, and delete any already-approved post or reply, from the Admin tab. See 10.3/10.4 for the updated Post status field and the new /api/discussions/moderation/pending, /posts/:id/approve, and DELETE /posts/:id endpoints (and reply equivalents).

11.7 Known limitations of the current setup

- **Render free tier cold starts.** The backend spins down after ~15 minutes of inactivity; the first request after idling takes 30–50 seconds to wake it back up. Acceptable for a demo/early-access stage, not for production traffic expectations.
- **Placeholder credentials.** The seed admin account (caregiverplatform@gmail.com) and the four test personas use known passwords and must be rotated or removed before onboarding real users (see 11.5).
- **No CI/CD gate.** Both platforms deploy directly off a push to main with no automated test run in between.